

1. O que o exemplo da imagem da Aula 02 está pedindo?

A proposta é implementar uma função:

```
public int ultimo(LinkedList L, int key)
```

que percorra uma **lista encadeada** e encontre **o último nó que possui determinada chave (key)**.

Por exemplo, considere:

Lista:

[10] → [25] → [10] → [40] → [10] → NULL

Se:

key = 10

a função deve retornar o **último nó que contém 10**:

[10] → [25] → [10] → [40] → [10]

↑

último 10

2. Como poderíamos implementar?

Uma solução simples seria:

```
LinkedList ultimo(LinkedList L, int key) {
```

```
    LinkedList ultimo = NULL;
```

```
    while (L != NULL) {
```

```
        if (L->key == key) {
```

```
            ultimo = L;
```

```
        }
```

```
        L = L->next;
```

```
    }
```

```
    return ultimo;
```

```
}
```

A ideia é:

1. Começamos no primeiro nó.
 2. Verificamos se a chave é igual a key.
 3. Se for, guardamos aquele nó em ultimo.
 4. Avançamos para o próximo nó.
 5. Continuamos até chegar ao final.
 6. Como continuamos atualizando ultimo, ao final ele contém o **último nó encontrado**.
-

3. Por que precisamos percorrer toda a lista?

Essa é a parte mais importante para entender a **complexidade de algoritmos**.

Imagine:

10 → 20 → 30 → 40 → 50 → NULL

Queremos encontrar o último 10.

Encontramos 10 logo no primeiro nó.

Mas podemos parar?

Não.

Precisamos verificar os outros nós para descobrir se existe outro 10 mais adiante.

Por isso, mesmo que a chave apareça logo no começo, o algoritmo precisa percorrer a lista inteira.

Assim, para uma lista com n elementos:

nós examinados = n

Portanto:

$$T(n) = O(n)$$

A complexidade do algoritmo é **linear**.

4. O que significa O(n)?

O O(n) representa o crescimento do custo do algoritmo conforme aumenta o tamanho da entrada.

Imagine:

Tamanho da lista Nós analisados

10	10
100	100
1.000	1.000
10.000	10.000
1.000.000	1.000.000

Se dobrarmos o tamanho da lista, aproximadamente dobramos o trabalho.

Por isso chamamos de:

Complexidade linear — $O(n)$

5. E o que significa "custo de cada instrução"?

A imagem pede:

"Avalie o custo do algoritmo levando em conta o número de instruções e o custo de cada instrução."

Isso significa que não devemos simplesmente contar quantas linhas de código existem.

Devemos analisar **quantas vezes cada operação é executada**.

A imagem estabelece um modelo simplificado:

Operação	Custo
Comparação	1
Atribuição	2
Operação matemática	3

Esses valores são **custos hipotéticos** usados para exercitar a análise. Na prática, uma CPU não necessariamente executa essas operações com esses custos fixos.

6. Analisando o algoritmo

Considere novamente:

```
LinkedList ultimo(LinkedList L, int key) {  
    LinkedList ultimo = NULL;  
  
    while (L != NULL) {
```

```

    if (L->key == key) {
        ultimo = L;
    }

    L = L->next;
}

return ultimo;
}

```

Vamos analisar.

Inicialização

ultimo = NULL;

É uma **atribuição**.

Segundo o modelo da questão:

$$C = 2$$

Condição do while

$L \neq \text{NULL}$

É uma **comparação**.

Para uma lista com n elementos, essa condição será avaliada:

$$n + 1$$

vezes.

Por quê?

Porque temos:

$1^a \rightarrow \text{nó } 1$

$2^a \rightarrow \text{nó } 2$

$3^a \rightarrow \text{nó } 3$

...

$n^a \rightarrow \text{nó } n$

$n+1^a \rightarrow \text{NULL}$

Então:

$$C_{while} = n + 1$$

Comparação da chave

L->key == key

Essa comparação acontece uma vez para cada nó.

Portanto:

$$C_{comparação} = n$$

Como cada comparação custa 1:

$$C = n$$

Atribuição

Quando encontramos a chave:

ultimo = L;

temos uma atribuição.

O problema é que essa instrução **não necessariamente acontece n vezes**.

Ela acontece somente quando:

L->key == key

Se a chave aparecer k vezes:

$$C_{atribuições} = 2k$$

Avanço para o próximo nó

L = L->next;

É outra atribuição.

Ela ocorre para cada nó:

$$n$$

vezes.

Como custa 2:

$$C = 2n$$

7. E as operações matemáticas?

Neste algoritmo específico, praticamente não temos operações matemáticas como:

$i = i + 1;$

ou:

soma = soma + valor;

Portanto, utilizando exatamente o modelo apresentado no slide, **não há um custo de operação matemática relevante.**

Nem todo algoritmo possui necessariamente os três tipos de operações.

8. Custo total

Se considerarmos:

- ultimo = NULL $\rightarrow$ 1 atribuição $\rightarrow$ custo 2
- $L \neq \text{NULL} \rightarrow n+1$ comparações $\rightarrow$ custo $n+1$
- $L \rightarrow \text{key} == \text{key} \rightarrow n$ comparações $\rightarrow$ custo n
- ultimo = L $\rightarrow k$ atribuições $\rightarrow$ custo $2k$
- $L = L \rightarrow \text{next} \rightarrow n$ atribuições $\rightarrow$ custo $2n$

Temos:

$$T(n, k) = 2 + (n + 1) + n + 2k + 2n$$

Simplificando:

$$T(n, k) = 4n + 2k + 3$$

Como:

$$k \leq n$$

podemos considerar, no pior caso:

$$T(n) = O(n)$$

9. O ponto principal: complexidade assintótica

Perceba que, quando estudamos complexidade, não estamos tão interessados em saber se o algoritmo executa exatamente:

103 operações

ou:

203 operações

O mais importante é saber **como o custo cresce quando n aumenta.**

Por exemplo:

$O(1)$

10 elementos $\rightarrow \sim 1$ operação

100 elementos $\rightarrow \sim 1$ operação

1.000 elementos $\rightarrow \sim 1$ operação

Custo constante.

$O(n)$

10 elementos $\rightarrow \sim 10$ operações

100 elementos $\rightarrow \sim 100$ operações

1.000 elementos $\rightarrow \sim 1.000$ operações

Custo linear.

$O(n^2)$

10 elementos $\rightarrow \sim 100$ operações

100 elementos $\rightarrow \sim 10.000$ operações

1.000 elementos $\rightarrow \sim 1.000.000$ operações

Custo quadrático.

10. Uma observação importante sobre "melhor caso"

Esse exercício é interessante porque podemos discutir **melhor, médio e pior caso**.

Para encontrar **o último nó** que contém uma chave, mesmo que encontremos a chave no primeiro nó:

[10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ [40] $\rightarrow$...

↑

encontrou!

não podemos parar.

Pode existir outro 10 depois:

[10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ [40] $\rightarrow$ [10]

↑

último elemento

Logo, nesse problema específico, precisamos chegar ao final da lista.

Portanto:

Caso	Complexidade
------	--------------

Melhor caso	$O(n)$
-------------	--------

Caso médio	$O(n)$
------------	--------

Pior caso	$O(n)$
-----------	--------

Encontrar um elemento qualquer pode permitir parar antes.

Encontrar o último elemento que possui determinada chave exige percorrer toda a lista.

11. Comparando com uma busca comum

Imagine agora uma função que procura **o primeiro nó** com determinada chave:

```
LinkedList buscar(LinkedList L, int key) {
```

```
    while (L != NULL) {
```

```
        if (L->key == key) {
```

```
            return L;
```

```
        }
```

```
        L = L->next;
```

```
    }
```

```
    return NULL;
```

```
}
```

Nesse caso, podemos parar imediatamente:

10 → 20 → 30 → 40 → 50

↑

key encontrada

Busca do primeiro:

Melhor caso:

$O(1)$

Pior caso:

$$O(n)$$

Já a busca pelo **último**:

Melhor caso:

$$O(n)$$

Pior caso:

$$O(n)$$

A imagem está ensinando basicamente este raciocínio:

LISTA ENCADEADA

|



Procurar o último nó

que possui determinada key

|



Precisamos percorrer

toda a lista

|



n elementos

|



Custo linear

|



$O(n)$

E a ideia central da **complexidade de algoritmos** é:

Não basta saber se um algoritmo funciona. Também precisamos analisar quanto trabalho ele realiza à medida que o tamanho da entrada aumenta.

Nesse exercício, quanto maior a lista, maior proporcionalmente será o número de nós que precisamos visitar. Por isso, a solução possui **complexidade temporal $O(n)$** .